Generative AI and LLM-based Agents

Fundamentals of Digitalization and Digital Trends — Lecture 01

Prof. Dr. Michael Granitzer

14 June 2026

Speaker notes

Welcome. This is an interdisciplinary lecture — a few of you come from computer science and AI, but most of you study social sciences, economics, or business. So I will not assume any machine-learning background. We build the whole story from the ground up.

The plan for the next 90 minutes: we start with what machine learning actually is, move to generative AI and how a transformer "thinks", then ask the hard question — is predicting the next word real reasoning or just a parrot? That leads us to agents (LLMs plus tools plus memory), and finally to the limits of AI and what that means for us as humans. Keep one question in mind throughout: generative AI is turning intelligence into a commodity you can buy by the token — how is that capability built, where does it reach its limits, and what does it imply for the future division of labour between humans and machines?

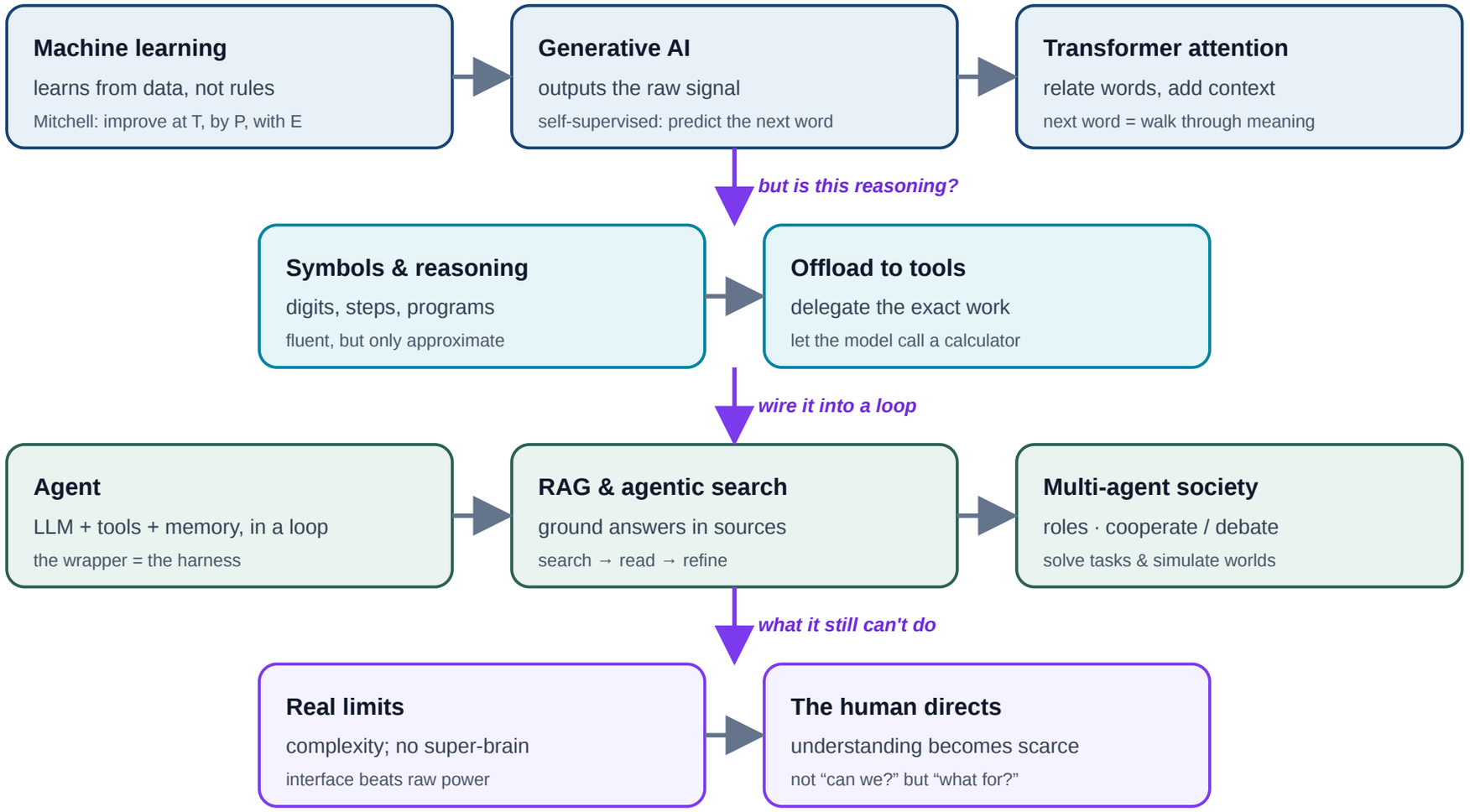
Where we are going

The question that frames this lecture: generative AI is making intelligence a commodity you can *buy by the token* — **how** is that capability built, **where** does it reach its limits, and **what** does that imply for the future division of labour between humans and machines?

Speaker notes

Hold that question — everything we build today is in service of answering it. The map on the next slide is our route: five stages, each an arrow, climbing from a program that learns from data up to systems that act in the world, and then stepping back to us. I want every non-technical student to leave able to explain each box to a colleague, and to have an informed opinion on the last one. Technical students: I keep the mechanisms light but true — nothing here is a lie-to-children you will have to unlearn. I will pause for questions at each module boundary.

Agendas



As raw intelligence becomes a commodity, value moves up — from doing the task to choosing and judging it.

The technology is only half the lecture; the other half is the question it forces on us — **follow the downward arrows.**

Speaker notes

This is the whole lecture on one slide — keep it as your map. Top row, *what the machine is*: learning from data, generative AI, the transformer. The second row turns prediction into action — symbols and tools. The third row is the agent and the multi-agent society. The bottom row is the counterweight, the real limits — which is exactly what leaves the human the job of understanding and direction. Each downward arrow is the question that carries us from one module into the next: "but is this reasoning?", "wire it into a loop", "what can it still not do?". By the end we return to this picture — and to the opening question.

1 — From Machine Learning to Generative AI

Guiding question: What does it mean for a machine to *learn*, and what is new about *generating* text and images?

Speaker notes

Module 1, roughly 30–32 minutes — the conceptual backbone of the lecture. We define machine learning, then build one picture of *what a learning system is* (X, F, Y, and the two places it can go wrong). We reuse that one picture for predictive and for generative learning, and then zoom inside the model — layers, attention — until "predicting the next word" becomes a walk through a map of meaning.

What is machine learning?

A classic, precise definition:

> "A computer program is said to **learn** from experience **E** with respect to some class of tasks **T** and performance measure **P**, if its performance at tasks in T, as measured by P, improves with experience E." — Mitchell (1997) (Mitchell 1997)

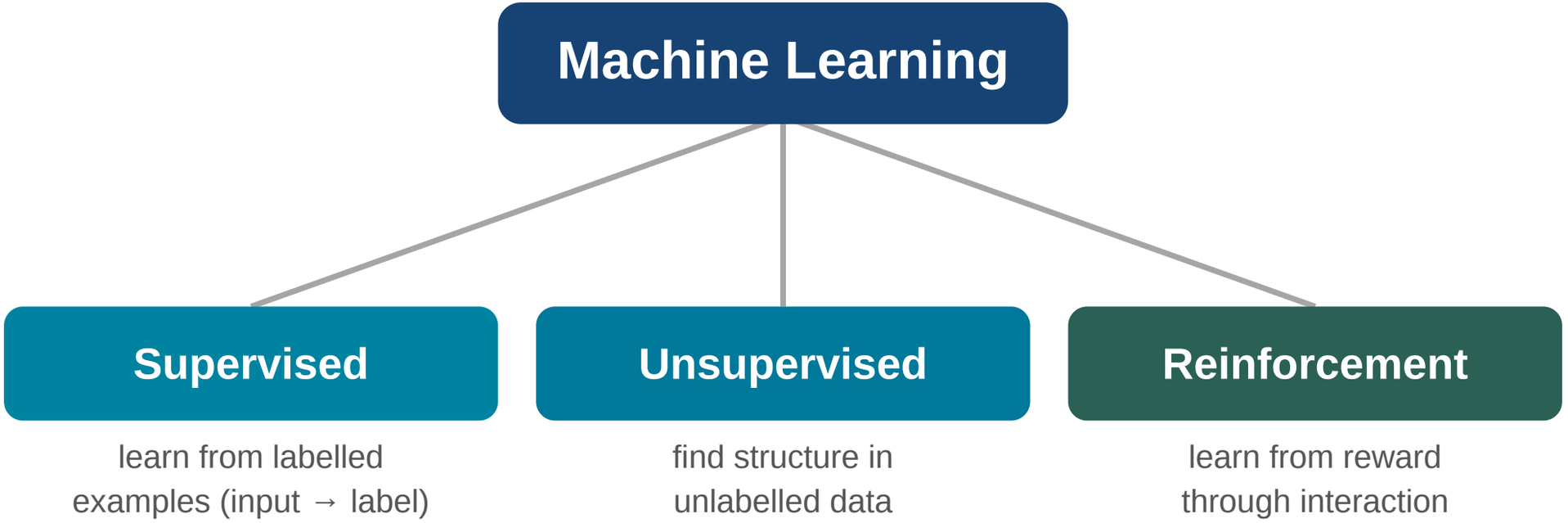
We do **not** write the rules by hand. We specify a task, a way to measure success, and let the program improve from experience, usually given in some form of data.

Speaker notes

This is Mitchell's definition and I like it because it is operational — it tells you exactly what to point at. T is the task, for example "predict tomorrow's demand". P is how you score it, for example forecast error. E is the experience — the historical data.

The shift from classical software is profound: in traditional programming a human writes the rules. In machine learning the human writes the *objective* and provides *examples*, and the rules are discovered. For the business and social-science students: this is why these systems are powerful and also why they are hard to govern — nobody wrote the rule, so nobody can simply read it off.

The main kinds of learning



Mitchell (1997): a program learns from experience E w.r.t. tasks T and measure P, if P on T improves with E.

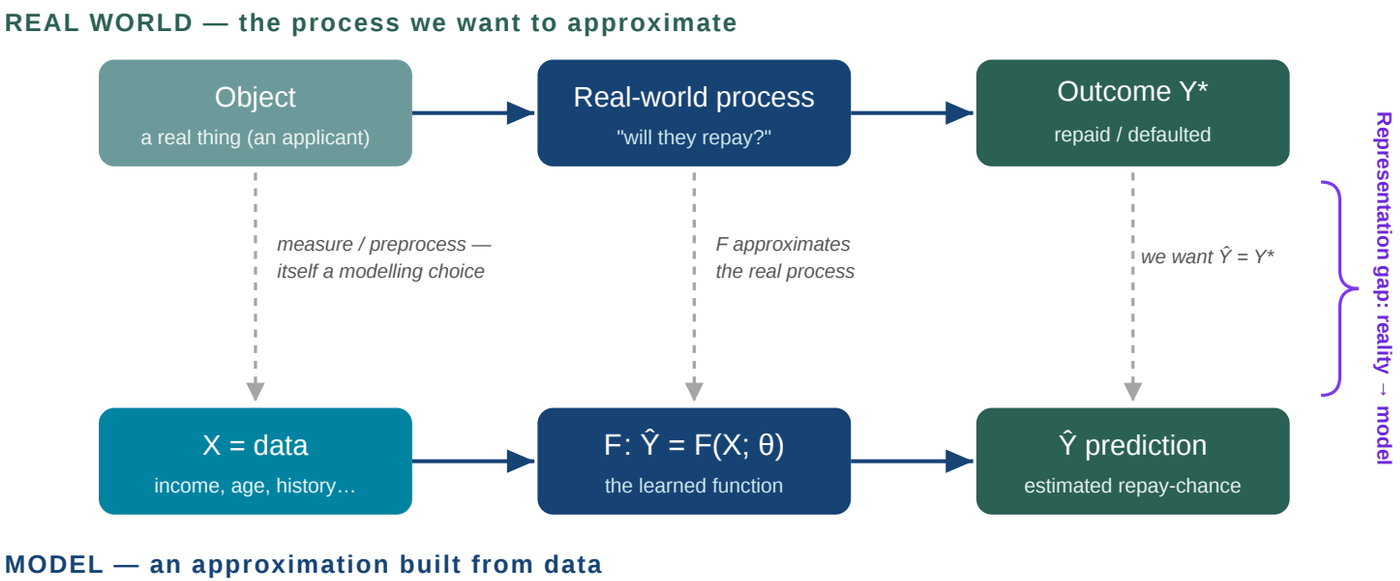
T — task **P** — performance **E** — experience

Speaker notes

Three big families. Supervised learning: you have labelled examples — emails marked spam or not — and learn the mapping from input to label. Unsupervised learning: no labels; you find structure — group customers into segments you did not know existed. Reinforcement learning: learn by acting and receiving reward, like a system learning to play a game or manage a portfolio.

Most of what made headlines — and everything today — grows out of the first two. And whatever the flavour, every learning system shares the same skeleton — which is the next slide.

From world to model



Both the data X and the function F are *our chosen models* of a messy reality.

A model is an **approximation of a real-world process**, written $\hat{Y} = F(X;\theta)$:

- X — the **data**: our chosen measurement of a real object *(preprocessing is already a modelling decision)*
- F — the **function** standing in for the real process; θ are its tunable **parameters** ("knobs")
- $\hat{Y}$ — the **prediction**; we want it close to the true outcome Y^*.
- For unsupervised problems Y^* is unkown and not available in the loss function

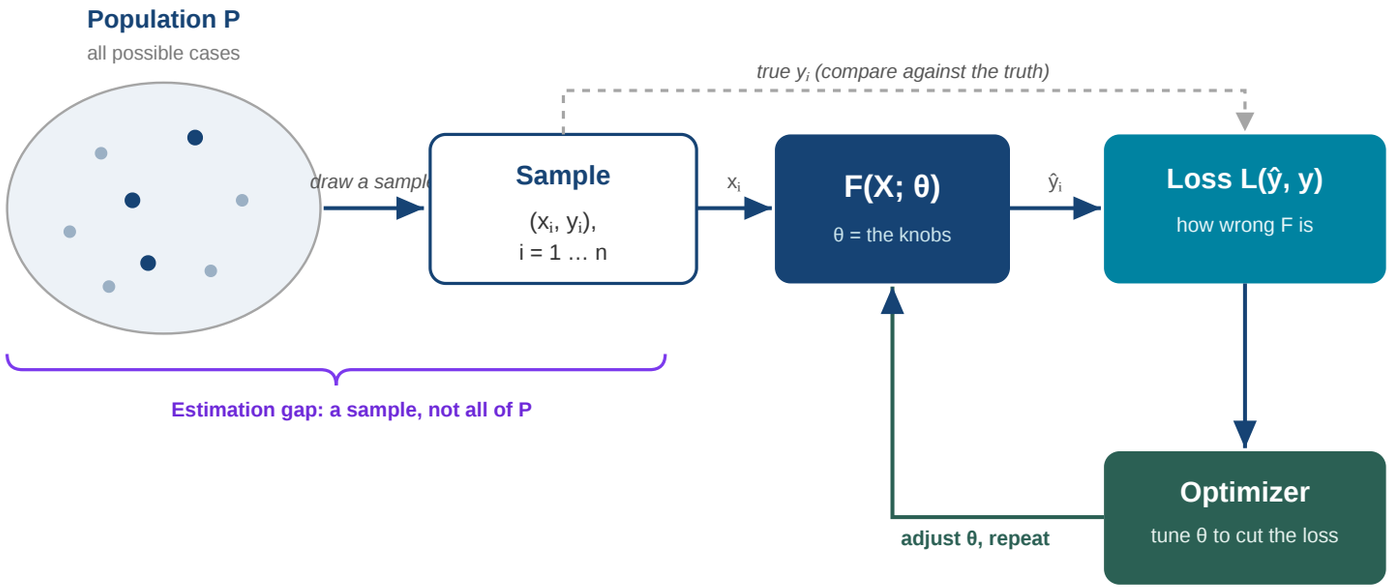
Abstracting a messy reality into X and F is the **representation gap** — even *what we choose to measure* is already a modelling decision.

Speaker notes

Here is the backbone of the whole module — one picture we will reuse. A machine-learning model is an approximation of some real-world process. Take lending: in the real world there is an applicant, a real process that decides whether they repay, and an actual outcome. We cannot capture that directly, so we build a model underneath. X is the data — but notice X is already a *choice*: we decide which features to measure — income, age, history — and how to encode them. That preprocessing is itself a model of the applicant. F is a function with tunable parameters theta — the knobs — standing in for the real process. Y-hat is its prediction, and we want it close to the true outcome Y-star.

The gap between the top row and the bottom row — between reality and our model — is the *representation gap*, and it never fully closes. For the social scientists in the room: this is already where bias can enter, in *what* we choose to measure and *how*.

Learning = estimating θ from a sample



$\theta^* = \arg\min_\theta \Sigma_i L(F(x_i; \theta), y_i)$ — pick the knobs that best fit the sample

We never see the whole world — only a **sample** (x_i, y_i) drawn from a population; that distance between sample and population is the **estimation gap**. Training **tunes the knobs** to fit the sample:

$$\theta^* = \arg\min_\theta \sum_i L\big(F(x_i;\theta),$$

The **loss** $L(\hat{y}, y)$ measures *how wrong* the model is; the **optimizer** turns the knobs to reduce it.

CLearning = estimatingonsequences of the gaps: the model can be **confidently wrong** on cases it never saw, and a **skewed sample → a biased model**.

Speaker notes

The second source of imperfection. Even granting the model, we never observe the whole population — only a sample of past cases. Learning means tuning the knobs theta so the model fits that sample: run each example through F, measure how wrong it is with a loss function, and let an optimizer nudge theta to reduce the total loss. That is all the formula says — pick the theta that minimises total wrongness on the sample.

Two consequences worth flagging now. Because it is only a sample, the model can be *confidently wrong* on cases it never saw — hold that for module 2. And if the sample is skewed, the model is skewed — the root of bias, a limit we return to in module 4. So: two gaps — reality-to-model, and population-to-sample. Everything else today sits on this one picture.

Predictive learning: the same picture, filled in

Here Y is a **label** we want to predict, and the **loss** scores *how wrong* each guess is — same skeleton, only X, the label, and the loss change:

Task	X (input)	F	Y (label)	Loss — *"how wrong?"*
Credit	income, age, history	classifier	repay? yes / no	a cost per wrong call — approving a defaulter hurts most
Spam	the email text	classifier	spam / not spam	count the misfiled emails
Vision	the pixels of a photo	classifier	cat / dog	share of photos mislabelled

The optimizer then tunes θ to make that loss as small as possible.

> The leap to generative AI is just a change in **what Y is** plus
>
> 1. a new kind of training algorithm: Transformers
> 2. massive amounts of data and corresponding infrastructure

Speaker notes

Let us make the skeleton concrete with three predictive — or "discriminative" — tasks, meaning Y is a label we want to guess. Same picture each time: an input X, the function F, a label Y. Credit: from an applicant's features, predict whether they repay. Spam: from the email text, predict spam or not. Vision: from the pixels, predict cat or dog. The machinery is identical; only X and the kind of label change.

The new column is the *loss* — simply a number for *how wrong* the guess was, the thing the optimizer drives down. Intuitively it is "count the mistakes": the share of emails misfiled, the share of photos mislabelled. But notice it need not weigh every mistake equally — in credit, approving someone who then defaults can cost far more than wrongly rejecting a good applicant, so we make that mistake more expensive in the loss. That choice of *what counts as wrong* is itself a modelling decision. Hold that thought, because the jump to generative AI is simply changing *what Y is* — which is the next slide.

Discriminative vs. generative

Same skeleton $\hat{Y} = F(X;\theta)$ — the difference is **what Y is**:

Discriminative — Y *is a label*

A verdict about the input.

- spam / not spam
- credit risk score
- cat vs. dog

Generative — Y *is the raw signal*

New content of the same kind as the data.

- write a paragraph
- draw an image
- compose audio

The leap of generative AI: it produces the **raw signal** — text, pixels, sound — not just a verdict about it.

Speaker notes

This is the conceptual heart of why generative AI felt like a discontinuity. For decades machine learning mostly *judged* things — it put inputs into boxes. A generative model instead *creates* an output in the same space as its training data: full sentences, full images.

Why is that so much harder and so much more useful? Judging needs you to capture the boundary between classes. Generating needs you to capture the whole structure of the data — grammar, facts, style, composition. A system that can generate fluent text has implicitly modelled an enormous amount about the world. That is also why it can be wrong so fluently, which we return to.

Generative learning in the same picture

A language model is just our skeleton with a special choice of X and Y:

- X = **the words so far**, and Y = **the next word** — so $\hat{Y} = F(X;\theta)$
- **self-supervised**: every sentence is its own training data — the next word *is* the label, so no human labelling is needed
- loss = "did it predict the actual next word?"; the optimizer tunes θ over **billions** of sentences

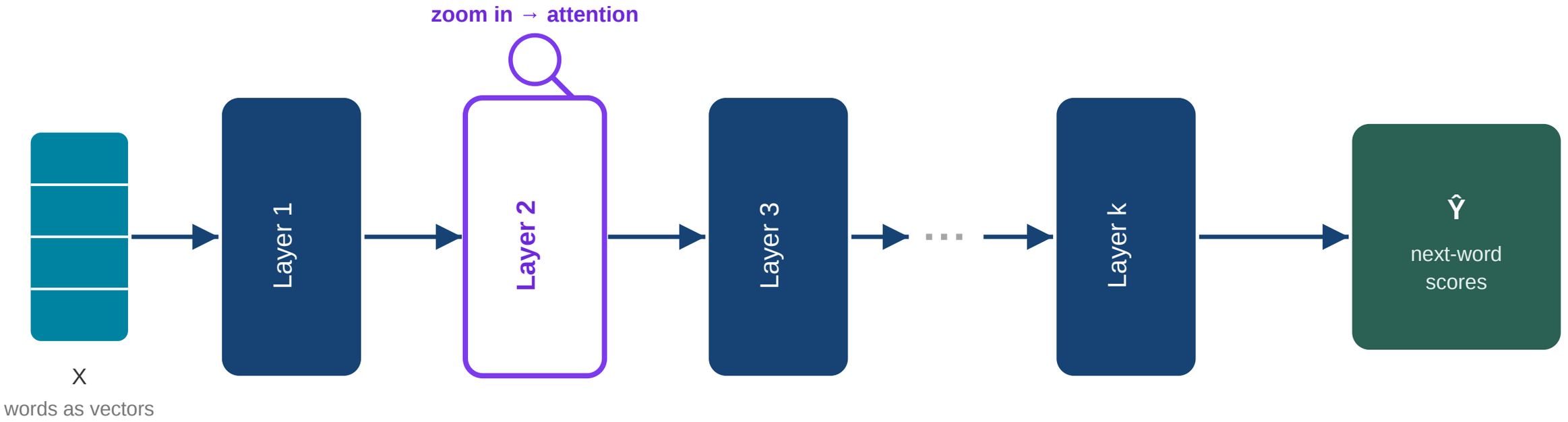
> Generate by repeating: predict the next word, append it, feed it back in. Consequently, language understanding seems to be a **supervised** learning problem.

So *what is inside* F that turns "the words so far" into a good next word?

Speaker notes

This is the pivot of the module. A generative language model is not a different kind of machine — it is exactly the learning system from the last three slides, with X = the text so far and Y = the next word. The beautiful part is *self-supervision*: nobody has to label the data, because the text labels itself — every position's correct answer is simply the word that actually came next. That is why these models could be trained on essentially the whole web. Train F to predict the next word well over billions of sentences, and to do that it has to absorb grammar, facts and style. The natural next question — and the rest of the module — is what is *inside* F.

Inside F: a stack of layers



$F = L_k \circ \cdots \circ L_2 \circ L_1$

each layer reshapes the representation a little

F is built from many simple **layers** stacked in sequence — $F = L_k \circ \cdots \circ L_1$. Each layer takes the current representation and **reshapes it** a little. Zoom into one layer and you find the transformer's key idea: **attention**.

Speaker notes

We have treated F as a single box. Open it up: F is a stack of many simple layers applied in sequence — that is what "deep" learning means, depth of layers. Each layer takes the current representation of the words and reshapes it a little, then passes it on. No single layer is clever; the power comes from stacking many. And if we zoom into what *one* layer actually does, we meet the transformer's central trick — attention — which is the next slide.

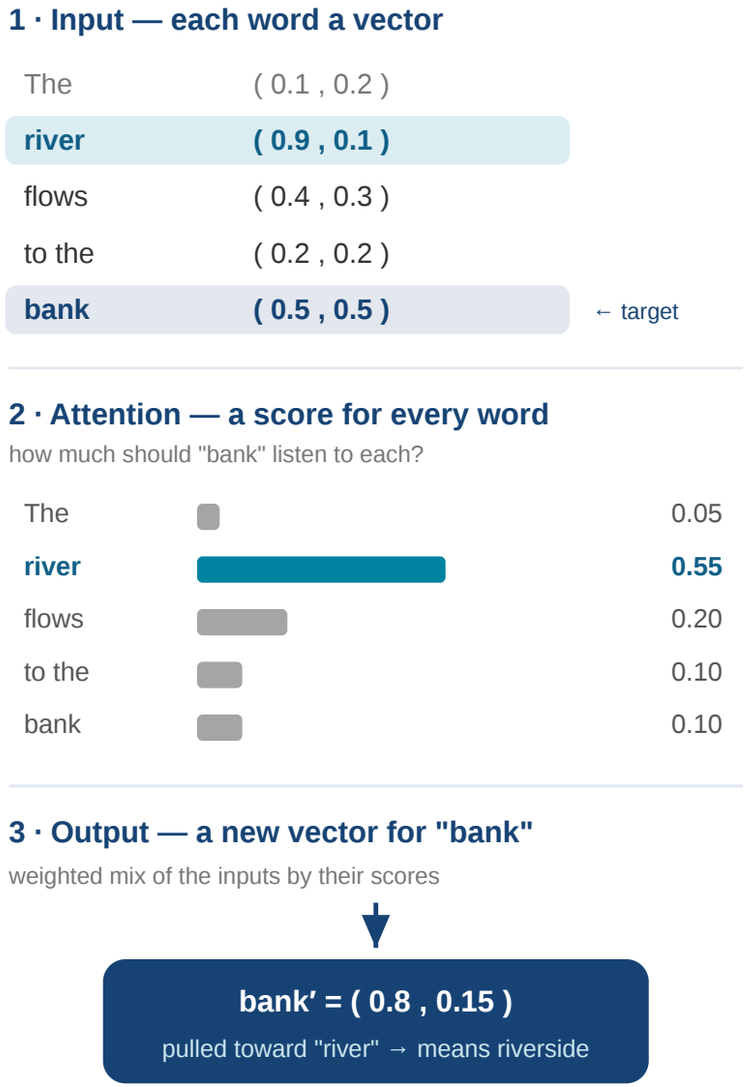
Attention: what one layer does

Zoom into one layer. For a target word it:

1. reads **all** the input words — each is a **vector** (a point in meaning-space);
2. **scores** how related every word is to the target — one number per word;
3. **attends**: takes a weighted mix of the word-vectors, by those scores;
4. outputs a **new vector** for the target — its context-aware meaning.

So the shape is **vectors in → scores → a new vector out**.

Here "bank" attends most to "river", so its output vector is pulled toward the *riverside* region of the map.

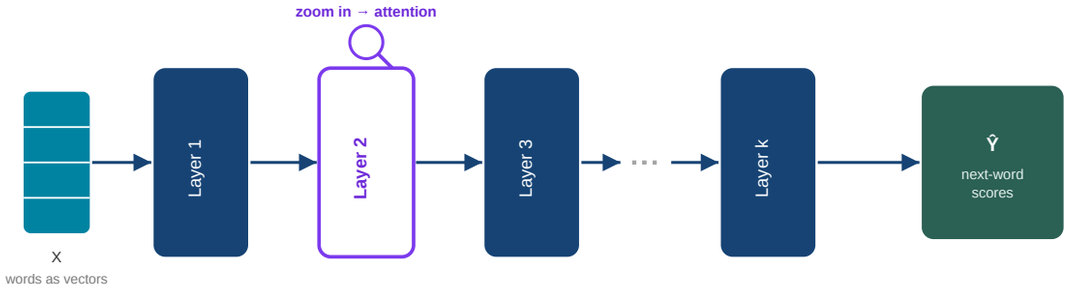


Speaker notes

Walk through the picture, which now has three steps. First, the input: every word enters as a *vector* — a handful of numbers, here just two so we can draw it. Second, attention is a *score*: for the target word "bank", the model rates how relevant every other word is — and note it scores *all* of them, not just a chosen few; "river" gets the highest weight, the rest little. Third, the output is again a *vector*: a weighted blend of the input vectors, so "bank" comes out moved toward "river" — it now means riverside, not a financial institution. Vectors in, scores in the middle, a new vector out. Vaswani and colleagues introduced this in 2017 (Vaswani et al. 2017); it is the engine under essentially every model you have heard of.

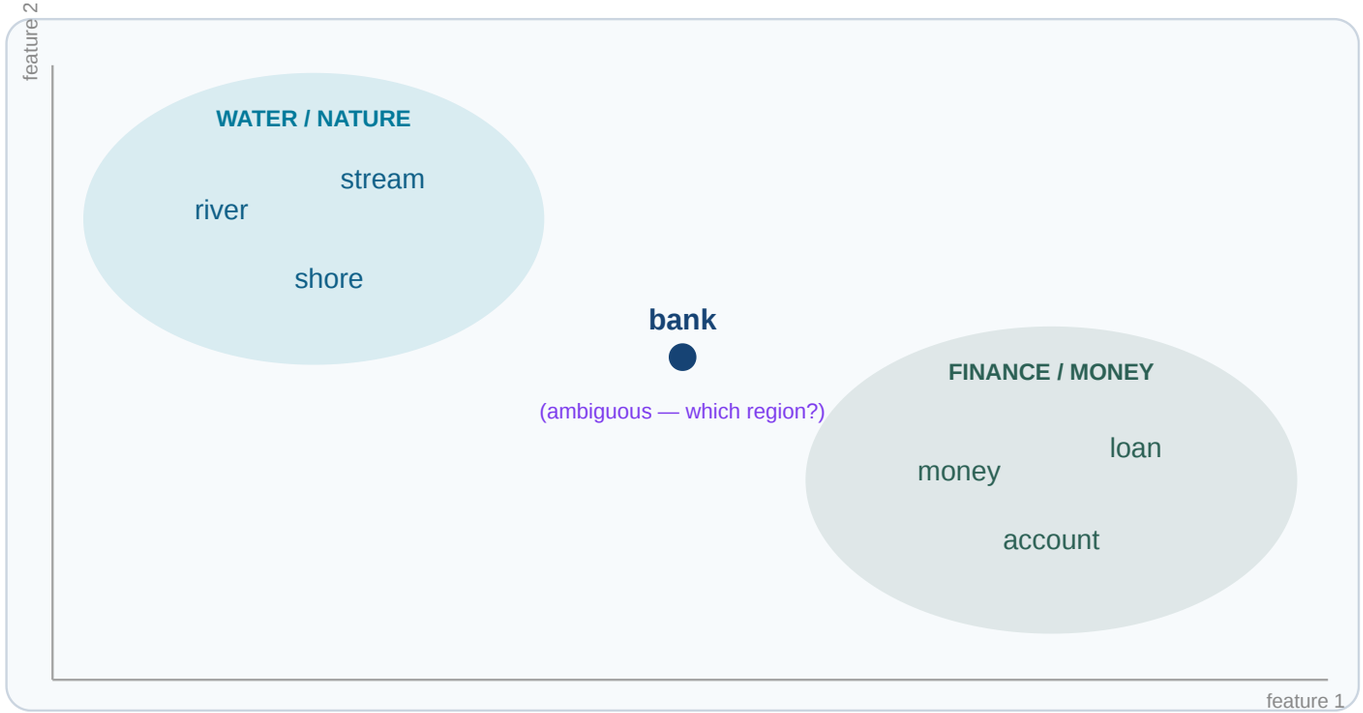
That is the whole idea — relate everything to everything, keep what is relevant, build a richer concept. Stack this operation many times (the "layers") and the representations get deep and abstract. Vaswani and colleagues introduced this in 2017 (Vaswani et al. 2017); it is the engine under essentially every model you have heard of.

Layers move points across a map of meaning



$F = L_k \circ \cdots \circ L_2 \circ L_1$

each layer reshapes the representation a little



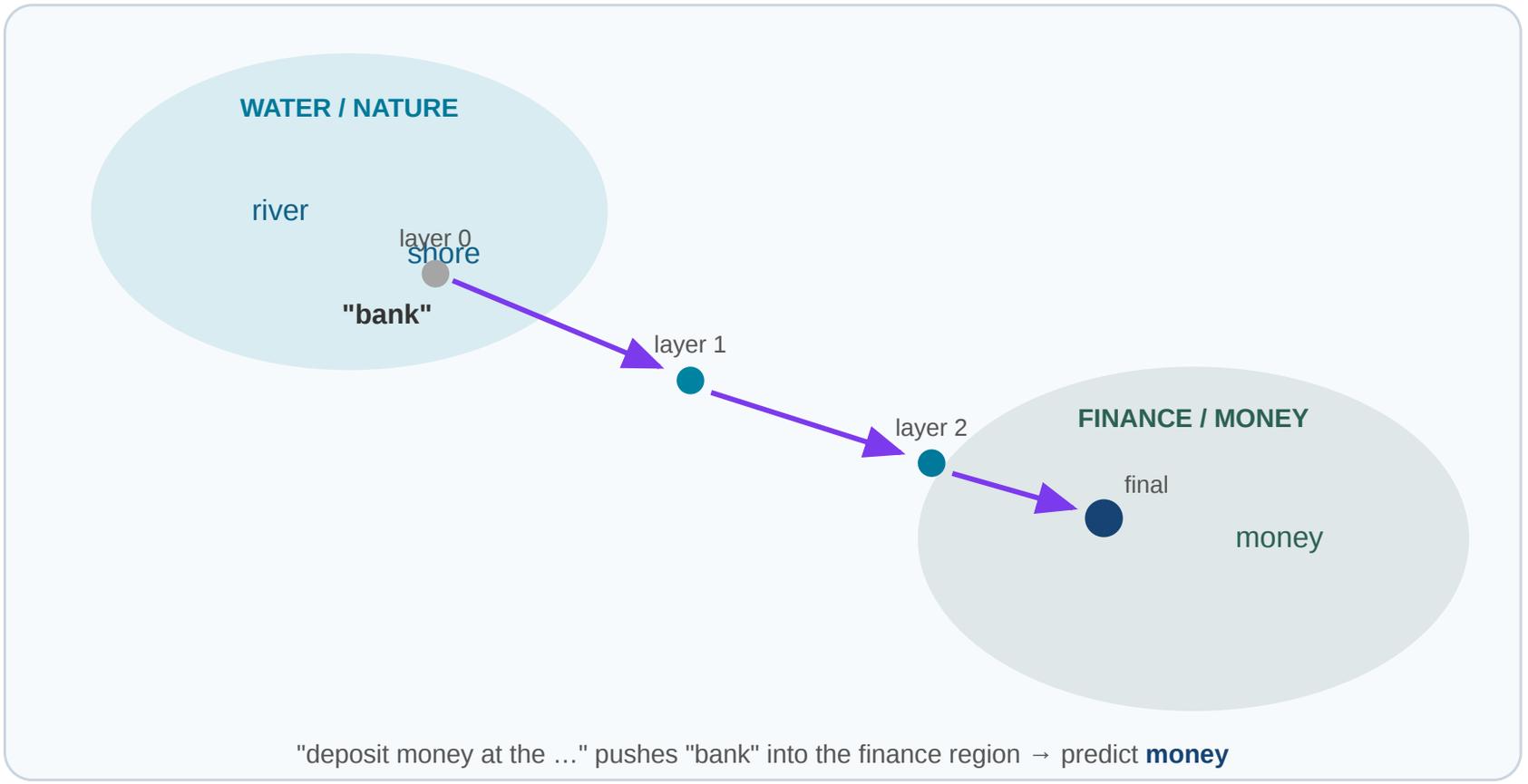
Picture every word as a **point on a map of meaning**, where related words sit close. **One layer = one step** that moves each point across the map.

Speaker notes

Now put the two pictures side by side, because together they are the payoff. On the left, the mechanism: a stack of layers, each reshaping the representation. On the right, the intuition: think of every word as a point on a map of meaning, where "river, stream, shore" cluster in one region and "money, loan, account" in another. This is not a metaphor I invented for class — word representations really do live in such a space, just in many more than two dimensions.

Put them together: one layer is one step that moves each point across this map. "Bank" starts out ambiguous, sitting between the two regions; the layers walk it toward one region or the other. The next slide animates that walk.

Each layer is a step across the map



"deposit money at the …" pushes "bank" into the finance region → predict **money**

Each transformer layer **nudges** every word's point across the map, using context. "Bank" starts ambiguous, between the regions; layer by layer the context *"deposit money at the …"* pushes it into the finance region.
After the last layer, we land in a region — and the **nearest words there** (money, account) are the model's prediction.

Predicting the next word is a **guided walk through a space of meaning** — context steers

the path, and where you land is what you say.

Speaker notes

This is the payoff of the module. Start with "bank" sitting between regions. The sentence is "deposit money at the …". Layer by layer, the context "deposit, money" pushes the point for the last position away from the nature region and into the finance region. When the walk ends, the model looks at where it landed and reads off the nearest words — "money", "account" — as the likely next token.

So "next-word prediction" is not a lookup table and not random. It is a trajectory through meaning, shaped by everything you have said so far. That picture is close to the truth and it will carry you a long way. Questions before we ask whether this counts as thinking?

2 — Computing with Symbols: reasoning or repetition?

Guiding question: Is "predict the next symbol" real reasoning, or a *stochastic parrot* (Bender et al. 2021) repeating patterns?

Speaker notes

Module 2, about 20 minutes. The critique you will have read in the press: these models are "stochastic parrots" — they remix what they have seen with no understanding. There is truth in it. But I want to complicate the picture with a concrete example, because the answer is more interesting than yes or no.

Two tasks — hard to just *know*

A · Arithmetic

Compute **2840 ÷ 6** — *exactly*.

B · A process

List **every step** to register at the university and enrol in *this* course.

Neither answer can be **recalled** — you must **work through it**, one correct step at a time.

Speaker notes

Two tasks, deliberately unalike. The first is pure arithmetic — a division you cannot do at a glance. The second is not arithmetic at all: it is a real-world procedure most of you have actually lived through — getting registered as a student and signed up for this very course. What they share is the point: in neither case can you simply *retrieve* an answer. You must *generate* it, step by correct step. Give the room a moment to try both.

Both are solved by *symbolic reasoning*

A · Symbolic calculation — 2840 ÷ 6

- 6×4 = 24, so 28 − 24 = 4
- bring down 4 → 44; 6×7 = 42 → 2
- bring down 0 → 20; 6×3 = 18 → 2
- ⇒ **473, remainder 2**

B · A process description

1. apply / register online
2. upload documents → matriculation
3. activate the campus account
4. enrol in *this course* in the system
5. confirm and pay the fee

In both we **manipulate symbols by fixed rules**, step by step — that is **symbolic reasoning**.

Speaker notes

Here is how we actually solve them — and the two solutions are more alike than they look. On the left, the division: we follow the long-division *algorithm*, a fixed sequence of symbol manipulations — multiply, subtract, bring down, repeat — until we are done. On the right, the enrolment: we write down a *procedure*, an ordered list of steps that must happen in the right order. Strip away the surface and both are the same activity: start from some givens and apply fixed rules to reach a result, one step at a time. That activity has a name — symbolic reasoning — and the next slide pins it down.

What *is* reasoning?

> **Reasoning** = start from **axioms** (accepted facts) and apply **rules of inference** to derive new, *guaranteed* conclusions.

- **Axioms:** "All humans are mortal." · "Socrates is a human."
- **Rule:** if *all A are B* and *x is an A*, then *x is B*.
- ⇒ **Conclusion:** "Socrates is mortal."

Three properties: it is **provable** (every step is shown), **correct** (if the axioms and rules are), but can be **costly** (some chains are very hard to find).

Speaker notes

Let us pin down what "reasoning" means, since we are about to ask whether machines do it. Classically it is this: you start from things accepted as true — the *axioms* — and apply *rules of inference* that preserve truth, deriving new conclusions step by step. The textbook example: all humans are mortal, Socrates is human, therefore Socrates is mortal. Three properties follow. It is *provable* — the conclusion comes with the chain of steps that justifies it; you can show your work. It is *correct* — if the axioms are true and the rules valid, the conclusion is *guaranteed*, not merely likely. And it can be *costly* — for hard problems, finding the right chain of steps can take enormous effort, even when each step is simple. Hold those three; they are exactly where machines differ.

Reasoning, more formally

> A **formal system** = **axioms** + **inference rules**. A **proof** is a finite chain of statements, each one an axiom or derived from earlier ones by a rule.

Socrates as a derivation — the rule is *modus ponens* (from A and "A → B", infer B):

```
∀x.  Human(x) → Mortal(x)      (general rule / axiom)
     Human(Socrates)           (fact)
──────────────────────────────────────────────────────
∴    Mortal(Socrates)          (by modus ponens)
```

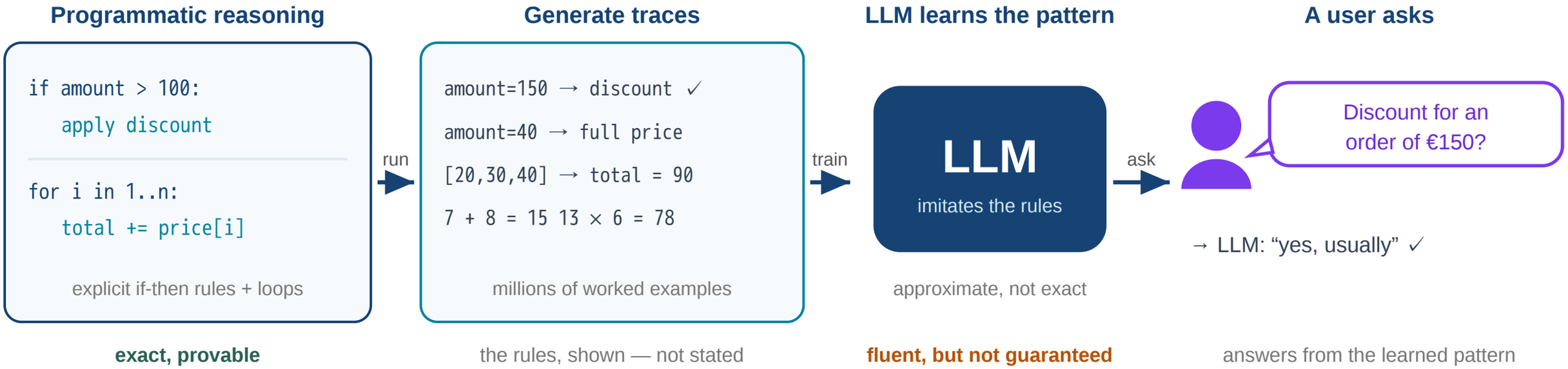
- **Sound** — true axioms + valid rules ⇒ every statement derived is *true*.
- **Costly** — finding a proof can be intractable: propositional satisfiability is already **NP-complete**; full first-order logic is **undecidable**.

Speaker notes

A little more formally, because some of you will want the real shape of it. A *formal system* is just two things: a set of axioms — statements taken as given — and inference rules — licensed ways to turn statements you already have into new ones. A *proof* is then a finite chain: each line is either an axiom or follows from earlier lines by a rule, and the last line is what you set out to show. The workhorse rule is *modus ponens*: if you have A, and you have "A implies B", you may write down B. The Socrates argument is exactly that — a universal rule "for every x, if x is human then x is mortal", the fact "Socrates is human", and one application of the rule. Two formal properties matter for us. *Soundness*: if the axioms are true and the rules valid, everything you derive is genuinely true — that guarantee is what symbolic reasoning buys you, and it is precisely what we will see machines lack. And *cost*: that guarantee is not free — even deciding whether a proof exists can be computationally explosive. Propositional logic is NP-complete; full first-order logic is undecidable, meaning no algorithm can always decide it. That tension — guaranteed but expensive — returns in module 4 when we hit the limits of computation.

Do LLMs reason?

Rules and programs **generate traces**; by predicting the next step, the LLM **imitates the pattern** — then answers new questions:



> But they do **not** run exact algorithms like a calculator. Anthropic's interpretability work finds a model adds numbers with **approximate, parallel heuristics** — not the carry algorithm — and then cannot faithfully report how it did it (Lindsey et al. 2025).

So it is **approximate reasoning**: often right and fluent, but **not guaranteed** — and oddly **expensive and unreliable** for exact work like adding large numbers.

Speaker notes

Walk the figure left to right. On the left, *programmatic reasoning* — explicit if-then rules and loops; run them and they are exact and provable. Run them on many inputs and you get the middle column: millions of worked *traces* — the rules *shown*, not stated. The LLM trains on those traces and learns to *imitate* the pattern — but approximately, not by re-running the program. So when the user on the right asks a new question, it answers from the learned pattern: fluent, usually right, but with no guarantee.

So do these models reason? The honest answer is "a little, and in a strange way." They have seen an astronomical number of reasoning *traces* — essentially every worked maths solution, proof and code snippet on the web — and by learning to predict the next step they have absorbed the *rules* well enough to reproduce them convincingly. That is genuinely more than parroting. But it is not the reasoning we just defined. Anthropic looked inside a model doing addition and found it does *not* run the algorithm a calculator runs: it uses a bag of approximate, parallel heuristics, lands near the right answer, and then — tellingly — gives a textbook explanation that is *not* what it actually did. So: approximate reasoning. Frequently correct and fluent, but with none of the guarantees — no proof, no certainty — and absurdly expensive and flaky for something a one-cent calculator nails every time. Which points straight to the fix on the next slide.

We offload exact work to a tool

Reasoning most often requires high effort.

So what do humans actually do? We **offload** exact symbolic work to a tool.

Brain	Calculator	Result
judgement, goals	exact arithmetic	fast & correct

> The skill that matters is not doing every step in your head — it is knowing **what to compute** and **delegating** it to something reliable.

Speaker notes

This is the bridge to the rest of the lecture. None of you would compute a long division by hand for real money — you reach for a calculator. You did not become stupid; you got *leverage*. You kept the part that is hard to delegate — deciding what needs computing — and handed off the mechanical part.

The same move rescues the language model from its arithmetic weakness: do not force it to do exact computation in its head. Let it *call a calculator*. A model that knows when to reach for a tool is no longer just a next-word predictor. But arithmetic is only the smallest case — the same idea scales up to whole processes, which is the next slide.

Symbols are not just digits

Arithmetic is symbolic — but so are **processes**, **procedures**, and **programs**.

Arithmetic	Process	Program
473 × 6 …	step 1 → step 2 → step 3	code that runs

The **same next-symbol machine** that predicts digits can predict the next **step of a workflow** or the next **line of code**.

> So an LLM can do more than call one tool — it can **write and drive a whole process**: a **controller of a workflow** that invokes many tools in sequence. That is an agent.

Speaker notes

This is the generalisation I want you to take from module 2. A calculation is a sequence of symbolic steps with a single right answer — but so is a business process ("receive order → check stock → invoice → ship"), an administrative procedure, or a computer program. All of them are *symbols arranged into a procedure*.

That matters because the very same next-symbol model can generate and follow such a procedure: the next step of a workflow, the next line of code, the next call to make. So the model is not limited to answering questions or calling one calculator — it can act as the *controller* of an entire process, deciding which tool to invoke at each step and feeding the result into the next. A next-word predictor wired up this way becomes a workflow controller. That is exactly the agent we build in module 3.

3 — LLM-based Agents

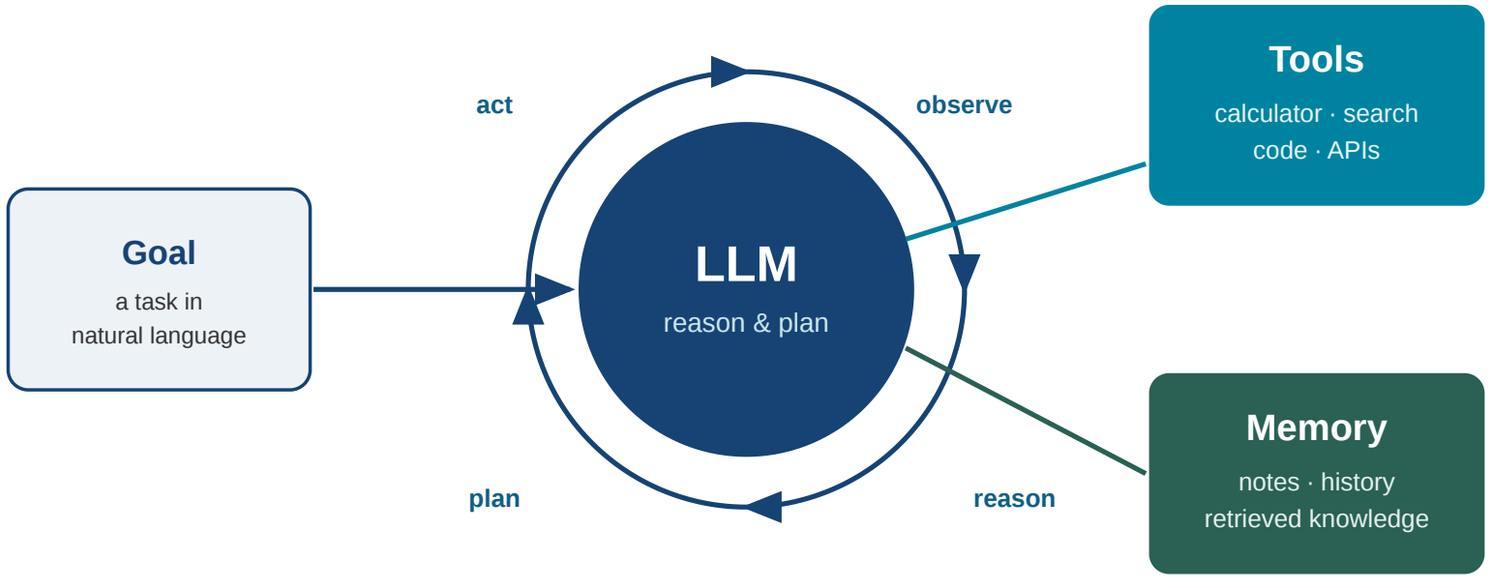
Guiding question: How does a text predictor become a system that *acts* — searching, computing, remembering?

📖 *Required reading: Guo et al. (2024), the multi-agent survey — this module maps directly onto it.*

Speaker notes

Module 3, about 28 minutes. This is the framing I used in a recent talk at the LRZ/TPC event, and it is the most useful definition I know. An agent is not a smarter model. It is a model wired to tools and memory inside a loop. We end on multi-agent systems and how they are used to simulate human worlds.

Agent = LLM + Tools + Memory



LLM

reason & plan

Tools

act in the world

Memory

carry context

The scaffolding that wraps the model — the **loop**, the **tools**, the **memory** and context management — is called the **agent harness**. The model does the reasoning; the harness

is everything else that turns a raw LLM into a working agent.

Speaker notes

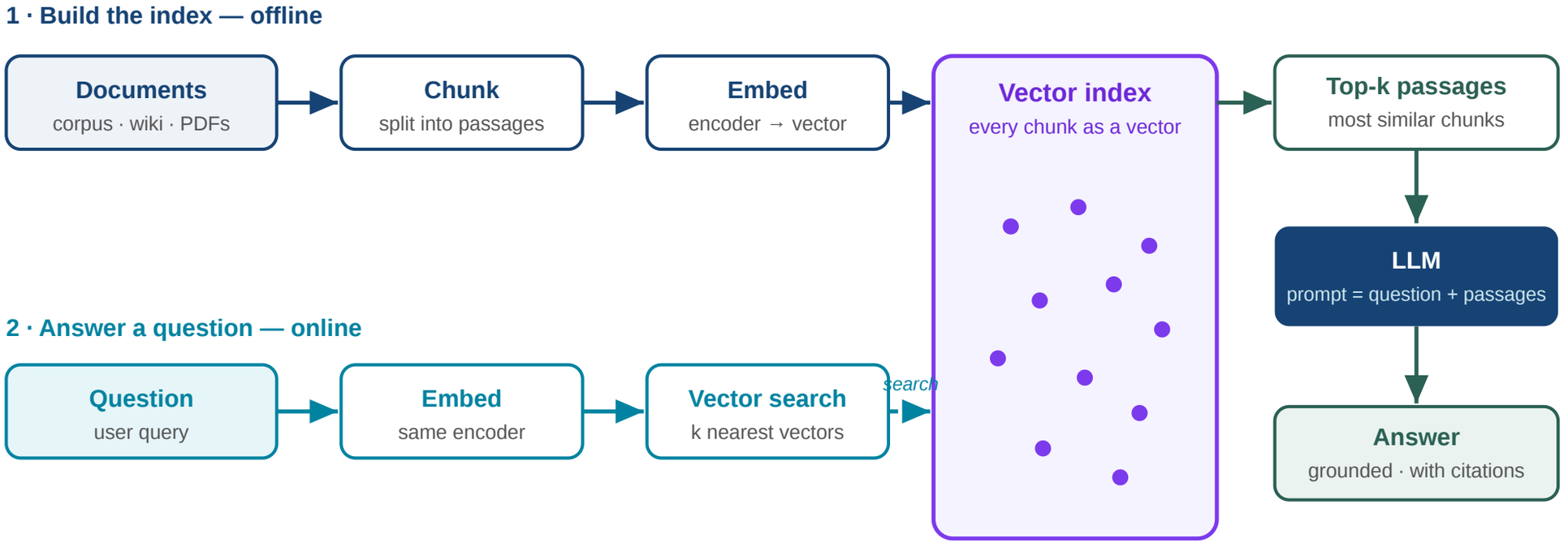
Three parts. The LLM is the reasoning core — it decides what to do next in words. Tools let it act: a calculator for arithmetic, a search engine for facts, a code interpreter, an API to send an email or query a database. Memory lets it carry information across steps — notes it took, results it got, knowledge it retrieved.

And it runs in a loop: observe the situation, decide on a step, act with a tool, read the result, repeat until the goal is met. The single model call becomes a little process — the LLM acts as the **workflow controller** we motivated in module 2, orchestrating tools step by step. This is the difference between "a chatbot answers" and "a system gets something done".

There is a name for all of this wrapper. The loop, the tool wiring, the memory and the context management around the model are collectively called the **agent harness** (some say *scaffolding*). Keep the distinction clear: the *model* supplies raw reasoning ability; the *harness* supplies the operational structure — what it can access, how it may act, what it remembers. The same model becomes a far more capable agent inside a better harness, which is why "harness engineering" is now a discipline of its own — and a useful idea for non-technical students too: when an AI tool disappoints, the bottleneck is often the harness around it, not the model inside it.

Giving the model knowledge: RAG

Retrieval-Augmented Generation — look things up *before* answering.



Vector search matches **meaning**, not exact words — it finds passages about the same idea even when they share no keywords (synonyms, paraphrase, other languages).

A pure model can **hallucinate**. RAG curbs this by **grounding** the answer in retrieved sources it can cite.

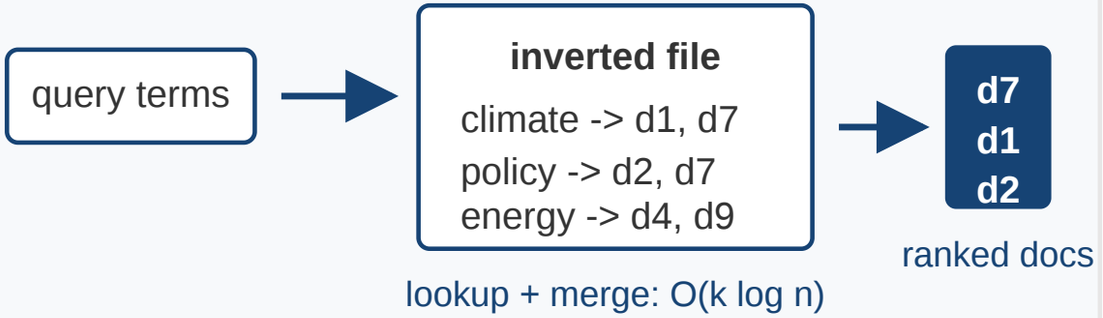
Speaker notes

Read the schematic as two lanes. Offline, the *indexing* lane: take your documents — a company wiki, a legal database, the web — split them into passages ("chunks"), run each chunk through an encoder that turns text into a vector, and store all those vectors in a vector index. Online, the *query* lane: embed the user's question with the same encoder, do a vector search to pull the top-k most similar chunks, drop those passages into the prompt next to the question, and let the LLM answer from them — grounded, and able to cite its sources.

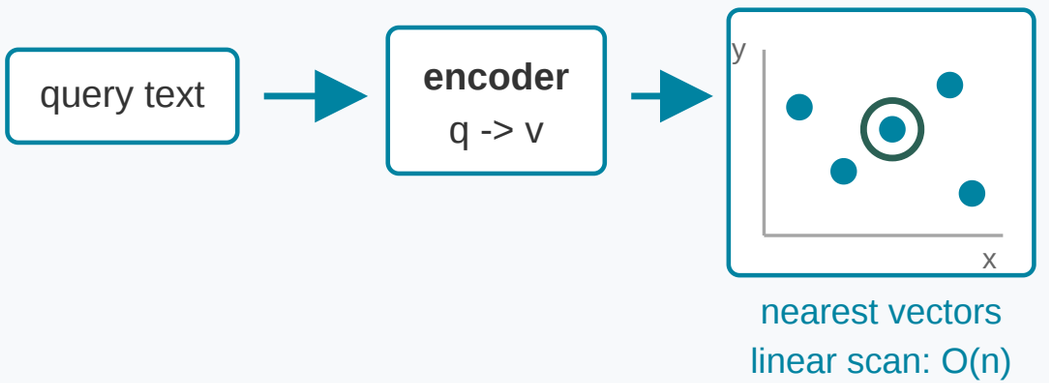
Two things to stress. First, the model is no longer answering from frozen training memory; it answers from evidence fetched at question time — cheaper, current, auditable, and the standard way enterprise AI works today (you don't retrain a giant model on your documents, you let it *retrieve* them). Second — the next slide — *why a vector index and not just keyword search?* Because the match is on meaning, not exact words.

Why vector search? keywords vs. meaning

Sparse index



Dense index



Sparse / keyword — an inverted file: the query word must *literally appear*. Fast, exact, but blind to wording.

Dense / vector — encode query and chunks into vectors; retrieve the **nearest** ones. Matches *meaning*, so it survives synonyms and paraphrase.

Speaker notes

Why bother embedding everything into vectors instead of plain keyword search? This is the linguistic-complexity problem. Classical search — the sparse, inverted-file approach on the left — needs the query's words to actually occur in the document. Ask for "car" and a page that only says "automobile" is missed; ask a question and a document that answers it in completely different words is invisible. Natural language is full of synonyms, paraphrase, and ambiguity, so literal matching is brittle. The dense approach on the right encodes both the query and every chunk into the same vector space — our "map of meaning" from module 1 — and retrieves the nearest vectors. Now "car" and "automobile" sit close, a question lands near its answer, and it even crosses languages. The cost is that exact nearest-neighbour search scans the whole index, so in practice we use approximate methods — but conceptually: sparse matches *words*, dense matches *meaning*, and meaning is what RAG needs.

From retrieval to agentic search

Plain RAG — *one shot*

retrieve once → answer.

Good for direct, single-fact questions.

Agentic search — *a loop*

search → read → refine the query → search again → synthesise.

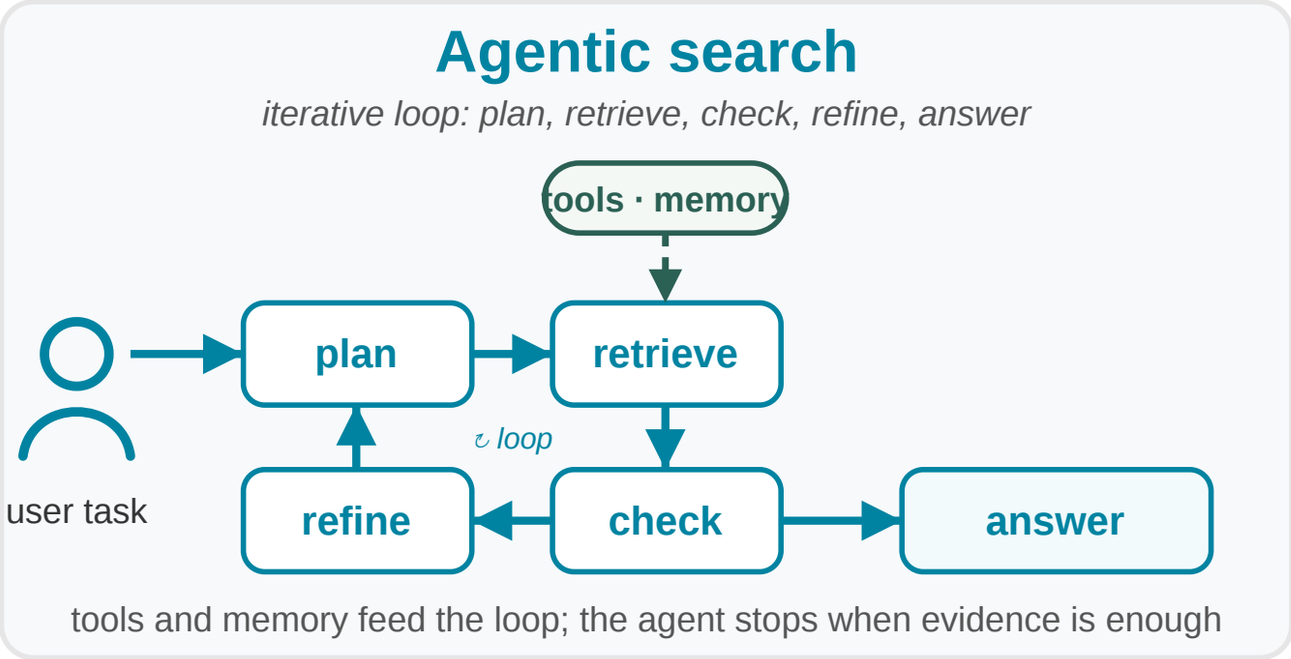
The agent decides *what to look up next*, based on what it just learned.

Speaker notes

Plain RAG retrieves once. But hard questions are not answered in one lookup — they need investigation. Agentic search puts retrieval inside the agent loop: the model searches, reads what it found, notices a gap, reformulates, searches again, and only then synthesises an answer. It behaves like a researcher working through a problem rather than a single database query.

This is the current frontier of search and "deep research" tools: the model is not just answering from one retrieval, it is conducting an inquiry.

Agentic retrieval: the workflow



Search becomes an **investigation**, not a single lookup:

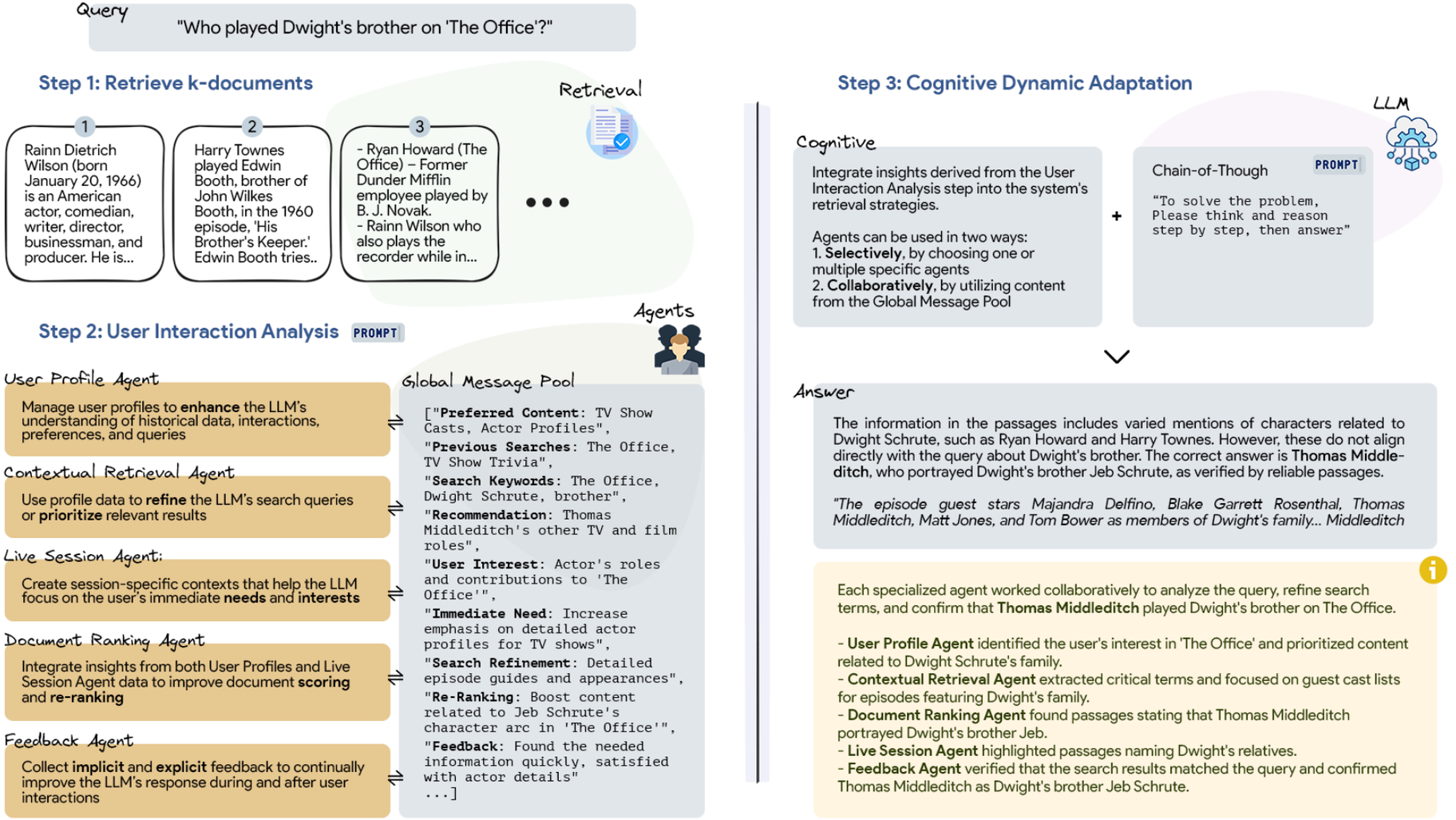
- **plan** the next sub-question
- **retrieve** with search + tools
- **check** the evidence so far
- **refine** and search again
- **answer** with provenance, once it is enough

> Retrieval inside a control loop — the agent decides *what to look up next*, and tools + memory feed every step.

Speaker notes

Here is the loop concretely — a figure from our open-web-search work. A user task comes in on the left; instead of one retrieval, the agent runs a cycle: plan a sub-question, retrieve with search and tools, check the evidence it has, refine and go again — and only emits an answer, with provenance, once the evidence is sufficient. The dashed link reminds us that tools and memory feed the loop at every turn. This is exactly the agent loop from two slides ago, with *retrieval* as the tool — which is why "deep research" assistants behave like a junior analyst rather than a search box.

PersonaRAG — retrieval with user-centric agents



Source: Zerhoudi & Granitzer 2024

Plain RAG retrieves for a *query* — not for a *user's* intent and history.

PersonaRAG wraps retrieval in a small society of **user-centric agents**:

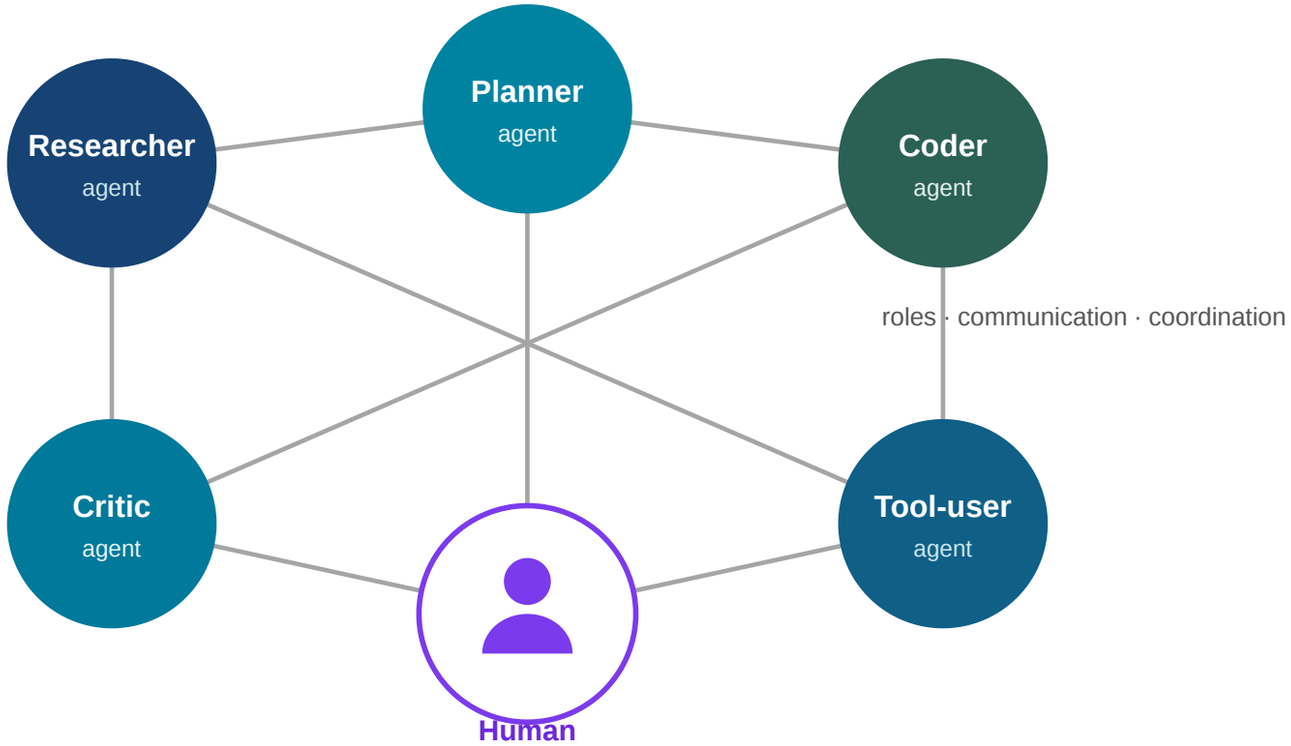
- user-profile agent
- contextual-retrieval agent
- live-session agent
- document-ranking agent
- feedback agent

The output is a **personalised evidence set**, not just a ranked list — **+10%** over vanilla RAG on WebQuestions (Zerhoudi & Granitzer 2024).

Speaker notes

A concrete research example from our group — PersonaRAG (Zerhoudi & Granitzer, 2024). The idea applies the multi-agent picture *to retrieval itself*: standard RAG looks up passages for the literal query, but a real user has preferences, a task, and an interaction history. PersonaRAG models those as a handful of in-context agents — a user-profile agent, a contextual-retrieval agent, a live-session agent, a ranking agent, and a feedback agent — that together shape what gets retrieved and how it is ranked. The retrieved set becomes *personalised evidence* rather than a generic ranked list, and across TriviaQA, WebQuestions and NaturalQuestions this lifts answer quality over both chain-of-thought and vanilla RAG. The wider lesson, and the bridge to the next module: once retrieval is agentic and personalised, *managing context* — what to keep, update, forget, and expose — becomes a first-class problem.

Many agents, not one



Multi-agent system can be organised along a few design choices (Guo et al. 2024) :

Profiling — give each agent a *role*: planner, researcher, coder, critic.

Communication — how they talk: *cooperate*, *debate*, or *compete*.

Capability — agents improve from *feedback* and shared *memory* →

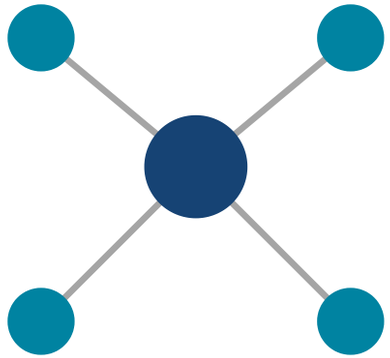
collective intelligence.

Speaker notes

Instead of one do-everything agent, use several specialised ones. The Guo et al. survey organises these systems along a few design choices. *Profiling*: each agent gets a role or persona — planner, researcher, coder, critic — either hand-designed or generated by the model itself. *Communication*: agents can cooperate toward a shared goal, debate opposing positions, or even compete. *Capability acquisition*: crucially, the agents need not be static — they improve from feedback (from the environment, from each other, from humans) and from shared memory. That is how a group of ordinary agents can reach a kind of *collective intelligence* none of them has alone — and it is the bridge to module 4.

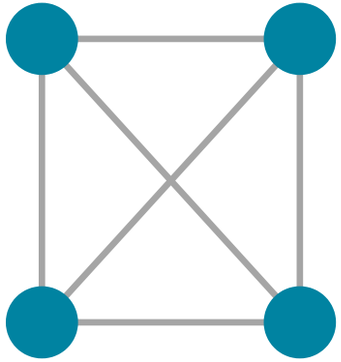
How agents coordinate

Centralized



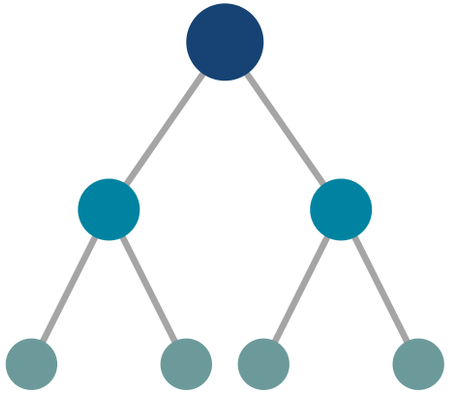
one coordinator

Decentralized



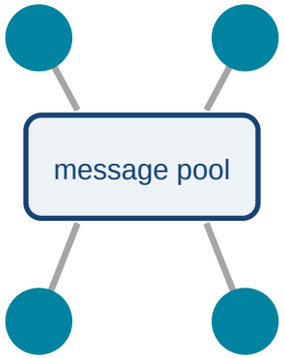
peer to peer

Layered



hierarchy

Shared pool



shared blackboard

Familiar from human organisations: a manager, a flat team, a hierarchy, or a shared workspace.

Cooperation — agents build on each other toward a shared goal.

Debate — agents argue opposing views; the clash *surfaces errors* and improves the answer.

Speaker notes

How you wire the agents together matters as much as how clever each one is — and the options are familiar, because they mirror human organisations. *Centralized*: one coordinator delegates, like a manager with a team. *Decentralized*: peers talk directly, like a flat team. *Layered*: a hierarchy, like a company org chart. *Shared pool*: everyone reads from and writes to a common "blackboard".

On the paradigm side, the two big ones are cooperation and debate. Debate is the surprising one: having agents argue opposing positions — the way a critic challenges a proposal — reliably catches mistakes a single confident agent would have kept. It is the multi-agent version of "show your work and have someone check it".

Simulating whole worlds

Beyond *solving* tasks, multi-agent LLMs are used to **simulate** them (Guo et al. 2024):

Societies	Economies	Populations
opinion & social-media dynamics	markets, trading, policy	epidemics, mobility

Give agents personas, let them interact, and study the **emergent** behaviour — a new instrument for the social and economic sciences. *(Example: Park et al.'s "Smallville" — 25 agents in a sandbox town that plan their days, gossip, and organise a party on their own (Park et al. 2023).)*

> Capability comes from **LLM + tools + memory + coordination** — and the same machinery becomes a *laboratory* for studying human systems.

Speaker notes

This is the half of the survey most relevant to your fields. Multi-agent LLMs are not only used to *get tasks done* — they are also used to *simulate* social and economic systems. Give each agent a persona, drop them into a simulated town, market, or social network, let them interact, and watch what emerges: how opinions spread, how a market clears, how a policy plays out, how a disease moves through a population.

For social scientists, economists and business students this is a genuinely new instrument — a sandbox where you can run "what if" experiments on human systems you could never run in reality. It also carries the obvious caveat: these agents are *models* of people, not people, so how far the results transfer is an open question — which leads us straight into the limits module.

4 — Limits and Human–AI Collaboration

Guiding question: What can AI *not* do for us — and what does that leave for humans?

Speaker notes

Final module, about 15 minutes. After all that capability, a sober counterweight: a handful of real limits — interface, complexity, no single super-brain, and the open challenges of multi-agent systems — and then a positive answer to "so what is left for us".

A better interface beats raw power

In *freestyle* (centaur) chess, teams of **amateurs with laptops and a good process** beat both grandmasters and supercomputers (Kasparov 2017).

> Kasparov's lesson: *weak human + machine + better process* > *strong human + machine + worse process*, and > *machine alone*.

The decisive variable was not horsepower — it was the **interface** between human and machine.

Speaker notes

Garry Kasparov — beaten by Deep Blue in 1997 — drew exactly the right conclusion. In freestyle chess, where any combination of humans and computers can compete, the winners were not the grandmasters and not the biggest machines. They were relative amateurs who had built an excellent *process* for working with their computers — knowing when to trust the engine, when to override, how to manage their time.

The lesson generalises far beyond chess. As raw model capability becomes a commodity, the advantage moves to whoever designs the best collaboration between human judgement and machine power. That is a design and process question — and it is good news for people who are not the single strongest expert in the room.

AI does not repeal hard problems

Some problems are hard for *anyone* — human or machine — because of their **computational complexity**.

> Even **approximate** Bayesian inference is **NP-hard** (Dagum & Luby 1993). Scaling a model bigger does not dissolve an NP-hard problem.

More compute makes many things faster. It does **not** abolish the wall that complexity theory puts in front of us.

Speaker notes

A crucial corrective to "AI will soon solve everything". Complexity theory tells us some problems are intractable in the worst case — the work grows explosively with size, for any method. Reasoning under uncertainty is a central AI task, and the result by Dagum and Luby shows that even *approximating* Bayesian inference is NP-hard. Not just exact inference — the approximation too.

The point for non-specialists: there is no amount of GPUs that turns an exponential wall into a stroll. AI gives us superb heuristics that work astonishingly well in practice, but "powerful AI" and "no hard limits" are different claims. The limits are real and mathematical.

There is no single super-brain

Look at human intelligence: it is **not one giant brain**. It is a **society** of brains — specialised, cooperating, correcting each other.

Unlikely: one monolithic super-intelligence that does everything.

Likely: a **society of agents and humans** — specialised, networked, cooperating (Guo et al. 2024).

Speaker notes

Connect the two halves of the lecture. We saw multi-agent systems work by cooperation, not by one omniscient agent. Human civilisation is the same: no individual holds all knowledge — science, markets, institutions are distributed systems of specialists who coordinate. Our greatest achievements are collective, not the output of one mega-mind.

So the most plausible future is not a single super-intelligence in a box. It is an ecosystem — many AI agents and many humans, specialised and networked, the way societies already are. That reframes the anxiety: the question is less "will one machine surpass us" and more "how do we design and govern this society of minds well".

...but that society is hard to build

The same survey is candid about the **open challenges** (Guo et al. 2024) — each a reason humans stay in the loop:

Errors compound — one agent's hallucination propagates through the group.

Collective intelligence — getting agents to genuinely *learn together* is unsolved.

Evaluation — what does "good" even *mean* for a whole society of agents?

Cost — many agents = many model calls = a lot of compute.

Speaker notes

The society-of-agents picture is promising, but the survey is honest about how far we are from making it reliable. Four open problems. First, errors *compound*: in a single model a hallucination is one wrong answer; in a multi-agent system one agent's mistake can be passed along and amplified by the others. Second, *collective intelligence*: getting agents to genuinely learn and improve together — continual learning as a group — is still largely unsolved. Third, *evaluation*: we barely know how to benchmark a single agent; judging a whole cooperating society is far harder, and without trustworthy evaluation we cannot rely on these systems. Fourth, *cost*: every agent is another model call, so a society of agents multiplies the compute and the bill.

Notice the through-line: every one of these is a reason to keep capable *humans* in the loop — to catch compounding errors, to judge quality, to decide whether the cost is worth it. Which is exactly the next slide.

So where does that leave the human?

In Andrej Karpathy's recent framing: **intelligence has become computable —** you can buy it by the token.

> What becomes scarce is **understanding** — the high-level grasp to *guide* intelligence toward the right ends.

The question shifts from **"can we do it?"** to **"what *should* we do?" —** the **goal**, not the skill.

Speaker notes

Andrej Karpathy's point, which I find the right note to end on. If raw problem-solving is now a utility you can purchase, then the bottleneck moves up a level. The scarce thing is no longer the ability to execute — it is understanding: knowing what is worth doing, framing the problem, judging whether an answer is good, taking responsibility for the consequences.

"Can we do X?" is increasingly answered by the machine. "Should we do X, and what exactly is X?" is ours. That is a shift from skill to goal, from execution to direction — and it needs creativity, judgement, and understanding, not faster arithmetic.

What to take away — and to build

> As intelligence becomes a commodity, your edge is **understanding and direction**: asking the right questions, setting the right goals, and judging the answers.

For an interdisciplinary world: the technical "how" is increasingly automatable. The "why", the "should", and the "what for" are yours — and they decide whether the technology serves us.

Speaker notes

Closing message, especially for the social-science, economics and business students. Do not feel you lost a race you were never in — the execution layer. Your training in judgement, ethics, institutions, human behaviour and value is exactly what the new bottleneck demands. The people who thrive will be those who can stand above the intelligence and steer it: frame problems, set goals, weigh trade-offs, and own the outcomes.

That is the human role in the age of generative AI and agents — not the smartest calculator in the room, but the one who decides what the calculation is *for*z.

Summary

Machine Learning → Generative AI → Symbols & Tools → Agents → a society of minds → the human as the one who *understands and directs*.

Speaker notes

To recap the arc. Machine learning is programs that improve from data, by Mitchell's definition. Generative AI is special because it produces raw signals, and a language model works by walking through a space of meaning, one context-steered step per layer. Predicting symbols is more than parroting but fallible — so we offload exact work to tools; and because processes and programs are symbolic too, the LLM can drive a whole workflow. Wire an LLM to tools and memory in a loop and you get an agent — a workflow controller; add retrieval and you get RAG and agentic search; add cooperation and you get multi-agent systems. But AI does not repeal complexity-theoretic limits, raw power loses to good human–machine process, and there will likely be no single super-brain — instead a society of agents and humans. Which leaves us the highest-value job: understanding and direction. Thank you — questions and discussion.

Discussion

- Where in **your** field is the bottleneck already shifting from *execution* to *understanding*?
- Which tasks would you happily delegate to an agent — and which would you never?
- If the future is a *society of agents and humans*, who is **accountable** when it errs?

Speaker notes

Open the floor. These prompts work across disciplines deliberately. Push students to name a concrete task in their own domain and locate the human-judgement layer in it. The accountability question is a good place to end, because it shows that the most important questions raised by this technology are not technical at all.

Image Credits & References

Lindsey, Jack and Gurnee, Wes and Ameisen, Emmanuel and Templeton, Adly and Batson, Joshua and others (2025). On the Biology of a Large Language Model. https://transformer-circuits.pub/2025/attribution-graphs/biology.html

Bender, Emily M. and Gebru, Timnit and McMillan-Major, Angelina and Shmitchell, Shmargaret (2021). On the Dangers of Stochastic Parrots: Can Language Models Be Too Big?. *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency (FAccT)*.

Dagum, Paul and Luby, Michael (1993). Approximating Probabilistic Inference in Bayesian Belief Networks Is NP-hard. *Artificial Intelligence*.

Guo, Taicheng and Chen, Xiuying and Wang, Yaqi and Chang, Ruidi and Pei, Shichao and Chawla, Nitesh V. and Wiest, Olaf and Zhang, Xiangliang (2024). Large Language Model Based Multi-Agents: A Survey of Progress and Challenges. *arXiv preprint arXiv:2402.01680*. https://arxiv.org/abs/2402.01680

Kasparov, Garry (2017). Deep Thinking: Where Machine Intelligence Ends and Human Creativity Begins. *PublicAffairs*.

Mitchell, Tom M. (1997). Machine Learning. *McGraw-Hill*.

Park, Joon Sung and O'Brien, Joseph C. and Cai, Carrie J. and Morris, Meredith Ringel and Liang, Percy and Bernstein, Michael S. (2023). Generative Agents: Interactive Simulacra of Human Behavior. *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*.

Vaswani, Ashish and Shazeer, Noam and Parmar, Niki and Uszkoreit, Jakob and Jones, Llion and Gomez, Aidan N. and Kaiser, Lukasz and Polosukhin, Illia (2017). Attention Is All You Need. *Advances in Neural Information Processing Systems (NeurIPS)*. https://arxiv.org/abs/1706.03762

Zerhoudi, Saber and Granitzer, Michael (2024). PersonaRAG: Enhancing Retrieval-Augmented Generation Systems with User-Centric Agents. *arXiv preprint arXiv:2407.09394*. https://arxiv.org/abs/2407.09394